Evidential Deep Learning for Handwritten Digit Recognition

A Recent Probabilistic (Dirichlet-Based) Classifier — CISC3024 Pattern Recognition, AI Assignment #2

Abstract

This report documents the end-to-end development of a recent, probability-based image classifier for computer vision, carried out entirely by an AI agent (Claude Code). The selected algorithm is **Evidential Deep Learning (EDL)** (Sensoy, Kaplan & Kandemir, NeurIPS 2018), which places a Dirichlet distribution over class probabilities so that every prediction is a second-order "opinion" with an explicit measure of uncertainty. A small convolutional network (105,866 parameters) with an evidential output head was trained on 2,000 MNIST images (500 for validation) and evaluated on 1,000 held-out official test images. It reached **91.00%** test accuracy and a macro-F1 of **0.9076**, compared with 55.60% for a Gaussian Naive Bayes baseline. The Dirichlet uncertainty decomposition behaves as theory predicts: misclassified images carry roughly twice the epistemic uncertainty of correct ones (0.3006 vs. 0.1508 nats). The complete, reproducible source code is available in the public repository linked at the end of this report.

1. Introduction

Assignment #2 of CISC3024 Pattern Recognition asks for a **recent** classification or recognition algorithm for computer vision and pattern recognition that is **based on probability or Bayes theory**. The task must be performed by an AI tool or agent across the entire workflow: searching for the algorithm, verifying it, programming, running experiments, analysing results, and writing the report — the student is not allowed to write any code by hand.

This report therefore has three goals: (1) to identify a genuinely recent, probability-based method rather than a classical shortcut; (2) to implement and evaluate it on a small, reproducible MNIST experiment on a consumer laptop (Apple M2, 8 GB RAM); and (3) to record, honestly, how the AI agent found, built, and tested the method.

2. AI-Assisted Algorithm Search and Selection

2.1 How the AI agent searched

The agent first formulated the requirement "recent + probability/Bayes theory + image classification + low compute" and searched the web for candidate methods, cross-checking each against its original publication (title, authors, year, and venue on the NeurIPS / ICML proceedings pages). Five candidates were shortlisted and compared, as summarised in Table 1. Only sources that could be verified against an original paper or official project page were kept; no reference was invented.

Table 1. Candidate algorithms investigated by the AI agent.

Venue	Probabilistic / Bayes basis	Decision

NeurIPS	Dirichlet distribution over class probabilities (subjective logic; conjugate prior of the categorical)	Selected
ICML	Dropout as approximate Bayesian inference (variational inference in a deep GP)	Rejected: more com
NeurIPS	Laplace approximation of the weight posterior	Rejected: extra bac
NeurIPS	Gaussian posterior approximation from SGD iterates	Rejected: more com
al	Bayes' rule with feature independence assumption	Rejected: not a rece

2.2 Why Evidential Deep Learning was selected

Gaussian Naive Bayes was deliberately not selected: although it is the easiest classical probabilistic classifier, it dates to the 1960s and does not satisfy the assignment's "recent" requirement, so it is used here only as a baseline. The fully Bayesian deep-learning alternatives (Monte-Carlo Dropout, Laplace Redux, SWAG) are recent and rigorous but require either multiple stochastic forward passes at test time, a Laplace/Hessian backend, or snapshot-covariance bookkeeping, all of which add complexity beyond what a small educational experiment needs.

Evidential Deep Learning is the **simplest method that genuinely satisfies every requirement**: it is recent (NeurIPS 2018); it is built directly on probability theory (the Dirichlet distribution, the conjugate prior of the categorical likelihood, via the theory of subjective logic); it classifies images with a standard convolutional network; and it needs only a single deterministic forward pass at inference time, with no Monte-Carlo sampling. Its uncertainty output is also directly useful for analysis. The added complexity of a Bayesian neural network was therefore not justified for this task.

3. Algorithm Theory

A standard classifier maps an input x to a single probability vector. EDL instead maps x to the parameters of a **Dirichlet distribution** over all possible probability vectors, so the output is a distribution over distributions (a second-order, or evidential, prediction).

For a K-class problem, the network outputs non-negative **evidence** $e_c \geq 0$ for each class (here via the softplus function). The Dirichlet concentration parameters are then

$\alpha_c = e_c + 1$, and the expected class probability is $p_c = \alpha_c / S$, $S = \Sigma_c \alpha_c$.

Intuitively, high evidence means the model has collected strong support for a class and is confident; low evidence means the model has little support and is uncertain. Because the Dirichlet is the conjugate prior of the categorical distribution, updating the belief with observed evidence follows the Bayesian rule of adding pseudo-counts — the +1 terms are the prior pseudo-counts.

Training maximises a **type-II maximum-likelihood** objective. For a one-hot label y and digamma function $\psi(\cdot)$, the per-sample loss is

$$L = \Sigma_c\, y_c \left(\psi(S) - \psi(\alpha_c) \right) + \lambda \cdot KL[\, \mathrm{Dir}(\tilde{\alpha}) \,\|\, \mathrm{Dir}(1) \,],$$

where the second term is the Kullback–Leibler divergence to a uniform Dirichlet (with the correct class's evidence removed in $\tilde{\alpha}$), regularising the model against

over-confident non-target evidence. The weight λ is annealed from 0 to 1 over the first epochs so that the data-fit term can establish itself first.

Because the prediction is a Dirichlet, its uncertainty can be decomposed (in nats):

- **Total uncertainty** = entropy of the expected categorical p = α / S.
- **Aleatoric uncertainty** = expected entropy of p under Dir(α) (irreducible, from the data).
- **Epistemic uncertainty** = total − aleatoric = the mutual information (reducible, from lack of evidence).

This is precisely what makes EDL satisfy the probability/Bayes requirement: it models uncertainty explicitly with a Dirichlet (a conjugate-prior Bayesian update), and its training objective is a marginal-likelihood-style objective over second-order predictions.

4. Dataset and Preprocessing

The experiment uses **MNIST** (LeCun et al., 1998): 28×28 grayscale images of the ten handwritten digits 0–9. MNIST is small, publicly available, and standard in pattern recognition, which keeps the experiment fast and reproducible.

- Training subset: 2000 images; validation subset: 500 images; test subset: 1000 images.
- A fixed random seed (42) drives the train/validation split, so the exact same images are chosen on every run.
- The test subset is taken from the official MNIST test split (first 1,000 examples, labels preserved), so no test image appears in training or validation.
- Pixels are scaled to [0, 1] by torchvision's ToTensor transform.
- For the Gaussian Naive Bayes baseline, each image is additionally flattened into a 784-dimensional vector.

5. AI-Assisted Implementation

The agent generated the whole project, organised as compact modules, then ran and debugged it autonomously:

- **model.py** — a small CNN (two 3×3 convolution + max-pooling blocks, then two linear layers) with an evidential head: evidence = softplus(logits), α = evidence + 1.
- **train.py** — the EDL loss (type-II ML term + annealed KL regulariser) and the Adam training loop, keeping the best-validation checkpoint.
- **dataset.py** — deterministic MNIST subsets with seed 42 and no test leakage.
- **evaluate.py** — accuracy, macro/per-class precision–recall–F1, confusion matrix, uncertainty decomposition, and the Gaussian Naive Bayes baseline.
- **plots.py** and **main.py** — figure generation and experiment orchestration; **config.py** records every hyperparameter.

Before the formal run, a sanity check verified the environment (Python 3.13 + PyTorch with the Apple Metal/MPS backend), loaded the subsets, checked

input/label/alpha shapes, ran one forward+backward pass, confirmed metrics were computable, and tested model save/reload. The formal run then completed in a few seconds on the M2 GPU.

6. Experimental Settings

All settings are recorded in results/config.json. The device was the Apple M2 GPU via the Metal Performance Shaders (MPS) backend.

Table 2. Experimental settings and hyperparameters.

Setting	Value
Hardware	Apple M2, 8 GB RAM, macOS (Apple Metal / MPS)
Random seed	42
Training / validation / test samples	2000 / 500 / 1000
Test-set provenance	Official MNIST test split (labels preserved, no leakage)
Model	Small CNN, 2 conv + 2 fully-connected layers
Parameters	105,866
Optimizer	Adam
Learning rate	0.001
Batch size	128
Epochs	20
KL annealing steps	10
Evidence function	softplus; alpha = evidence + 1

7. Experimental Results

Table 3. Overall test-set results (1,000 official test images).

Metric	EDL (CNN)	Gaussian Naive Bayes (baseline)
Test accuracy	91.00%	55.60%
Macro precision	0.9074	-
Macro recall	0.9085	-
Macro F1	0.9076	0.5259
Fit time (s)	4.387	0.0039
Inference time (s, 1,000 imgs)	0.0534	-

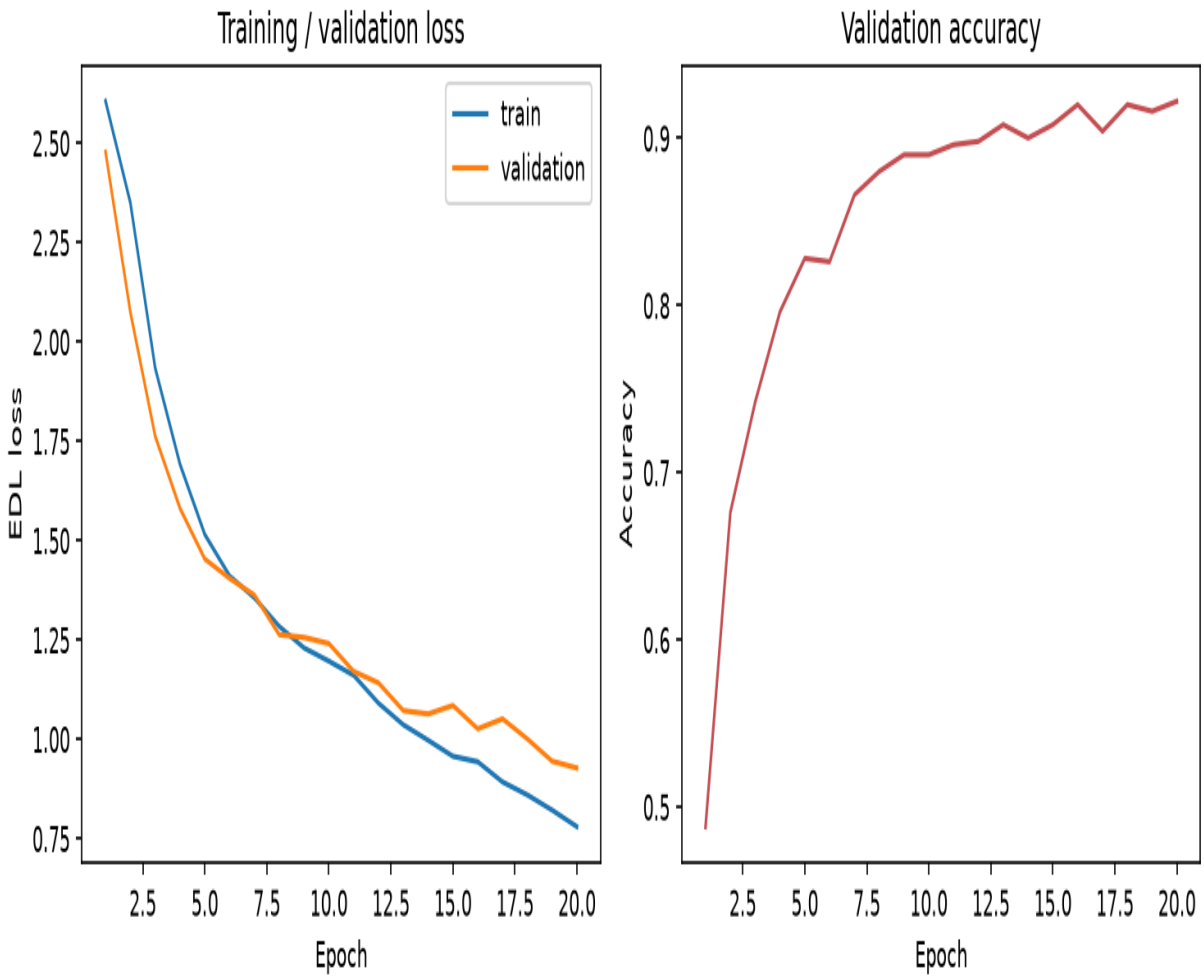


Figure 1. Training and validation EDL loss (left) and validation accuracy (right) over 20 epochs.

Training converged quickly: the EDL loss fell steadily and validation accuracy reached 92.20% at the best epoch (Figure 1). On the held-out test set the model achieved **91.00%** accuracy with macro-F1 0.9076, while the Gaussian Naive Bayes baseline reached only 55.60% (Table 3) — showing that the learned features carry information the independence assumption of Naive Bayes throws away. Training took 4.387 s and inference over all 1,000 test images took 0.0534 s.

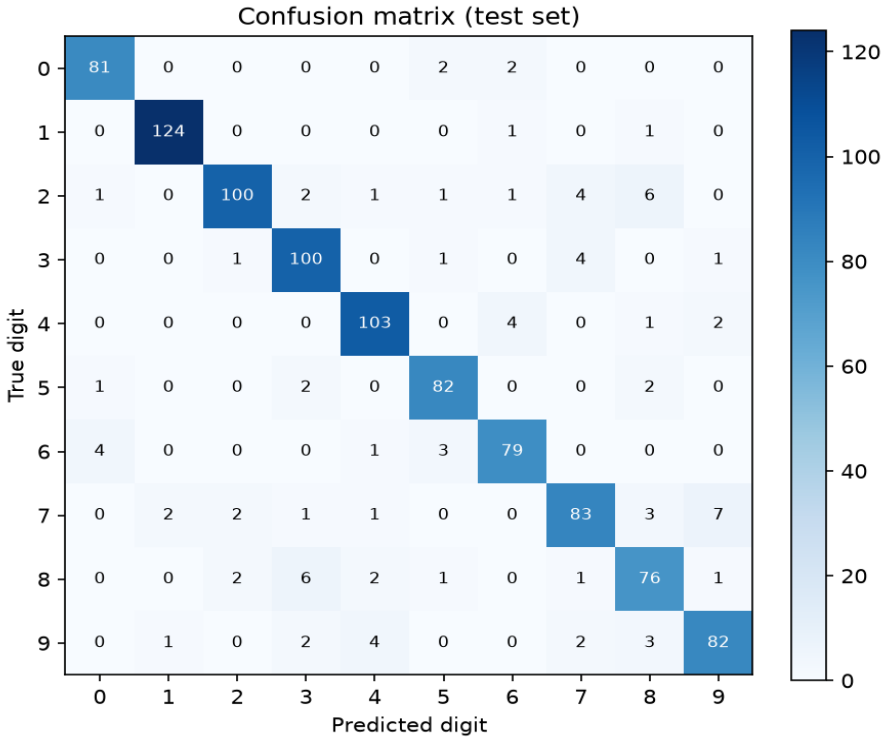


Figure 2. Confusion matrix on the 1,000-image test set.

Table 4. Per-class precision, recall and F1 on the test set.

Digit	Precision	Recall	F1
0	0.9310	0.9529	0.9419
1	0.9764	0.9841	0.9802
2	0.9524	0.8621	0.9050
3	0.8850	0.9346	0.9091
4	0.9196	0.9364	0.9279
5	0.9111	0.9425	0.9266
6	0.9080	0.9080	0.9080
7	0.8830	0.8384	0.8601
8	0.8261	0.8539	0.8398
9	0.8817	0.8723	0.8770

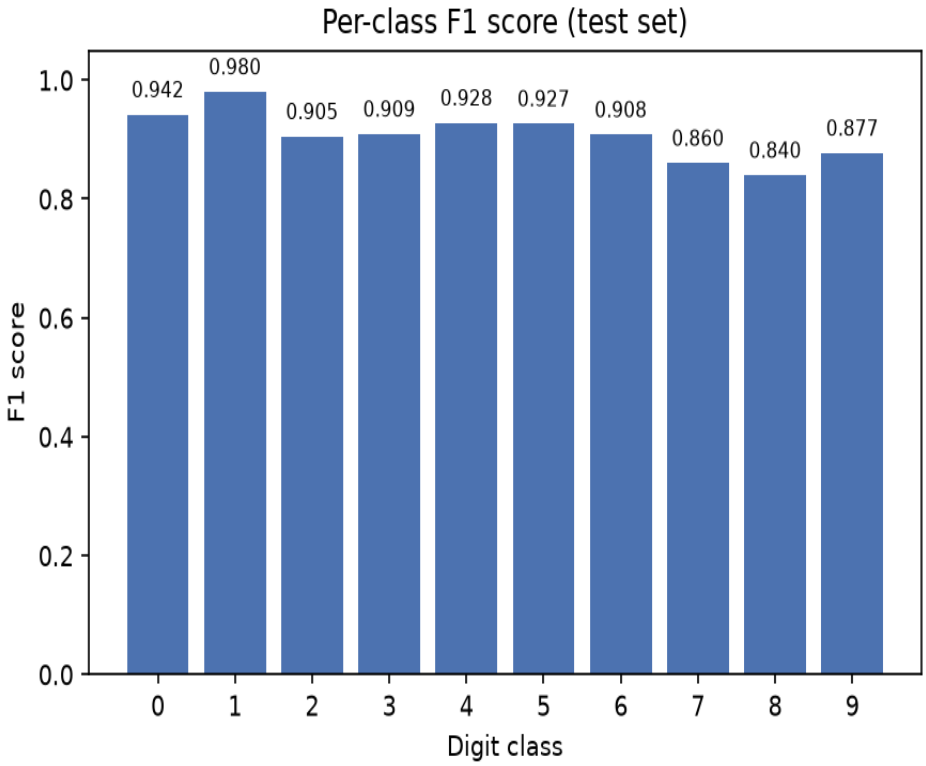


Figure 3. Per-class F1 score on the test set.

Per-class performance (Table 4, Figure 3) is strong and fairly balanced; the largest confusion is between visually similar digits such as 2–3–7–8, which is expected for handwritten digits.

Table 5. Mean Dirichlet uncertainty (nats) on the test set.

Uncertainty term	Mean (nats)
Total (entropy of p = alpha / S)	1.4435
Aleatoric (data / irreducible)	1.2792
Epistemic (model / reducible)	0.1643
Epistemic — correct predictions	0.1508
Epistemic — incorrect predictions	0.3006

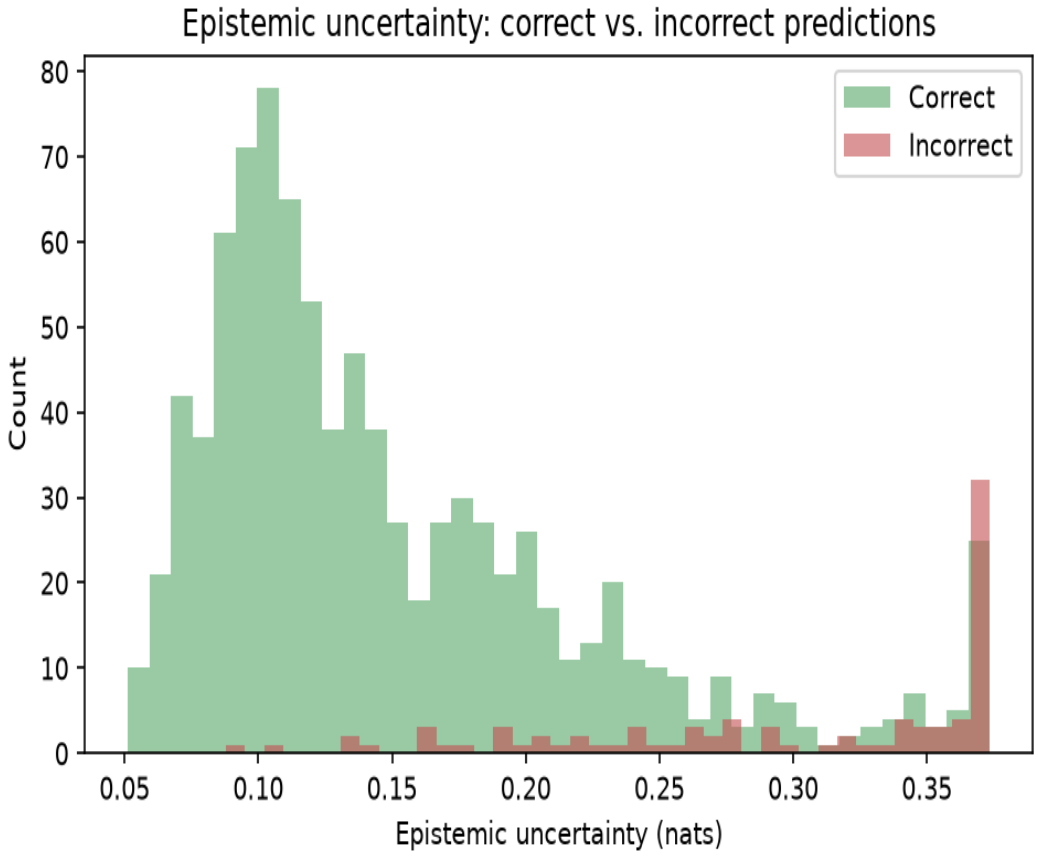


Figure 4. Epistemic uncertainty of correct vs. incorrect predictions.

The Dirichlet uncertainty decomposition (Table 5) is the most interesting result. Mean epistemic uncertainty is 0.1643 nats overall, but it is 0.3006 nats for misclassified images versus 0.1508 nats for correct ones — about a **two-fold** difference (Figure 4). In other words, the model tends to be more evidential-uncertain exactly when it is wrong, which is the intended behaviour of an evidential classifier and shows the uncertainty signal is informative rather than decorative. Aleatoric uncertainty (1.2792 nats) dominates epistemic uncertainty, as expected for clean in-distribution data.

Because these results come from a small 2,000-image training subset, they describe this specific reproducible experiment and should not be taken as the performance of EDL on the full MNIST dataset.

8. Discussion

EDL satisfies the assignment's core demand — a recent, probability-based classifier — while remaining lightweight enough to run in seconds on an 8 GB laptop. Its clear advantage over a classical Bayesian classifier (91.00% vs. 55.60%) is not a claim about EDL being "better than Bayes", but rather that a learned feature extractor combined with a Dirichlet output provides a much stronger inductive bias on raw pixels than the Naive Bayes independence assumption.

The uncertainty results support the method's claim to quantify uncertainty meaningfully: epistemic uncertainty tracks misclassification. At the same time,

recent work (Bengs et al., 2022) has shown that second-order losses such as EDL's do not, in general, provably recover faithful epistemic uncertainty, so the uncertainty numbers should be read as a useful empirical signal rather than a calibrated posterior. This nuance is acknowledged rather than hidden.

9. What I Learned from This AI Assignment

Technically, this assignment taught me how a Dirichlet distribution can serve as the conjugate prior of a categorical prediction, turning a single probability vector into a full second-order distribution with a principled split between aleatoric (data) and epistemic (model) uncertainty, and how a digamma-based marginal-likelihood objective can be trained end-to-end with a standard optimiser.

Methodologically, I learned that an AI agent can carry an entire research-and-engineering workflow — literature search, reference verification, code generation, debugging, experimentation, and report writing — but that its value depends on discipline: verifying every citation against an original source, pinning random seeds, recording the actual configuration, and refusing to fabricate numbers. Honest reporting of what actually ran (and what the small scale limits) is what makes the result trustworthy.

10. Limitations

- The training set is only 2,000 images, so results do not represent performance on the full MNIST dataset or on other datasets.
- A single random seed and no hyperparameter search mean the numbers are one point estimate, not a robust average.
- The network is deliberately small; a larger model would perform better but was unnecessary for this assignment.
- As noted, EDL's epistemic uncertainty is not guaranteed to be calibrated (Bengs et al., 2022).
- The experiment ran on one machine (Apple M2, MPS); timings are specific to that hardware.

11. Conclusion

Evidential Deep Learning (Sensoy et al., 2018) was identified as the simplest recent, genuinely probabilistic classifier suitable for a small educational experiment. Implemented as a small CNN with a Dirichlet output head, it reached 91.00% test accuracy on 1,000 held-out MNIST digits and produced an uncertainty measure that meaningfully separates correct from incorrect predictions. The entire workflow — from algorithm search to this report — was completed by an AI agent, with all results reproducible from the linked source-code repository.

12. References

- [1] Sensoy, M., Kaplan, L., & Kandemir, M. (2018). Evidential Deep Learning to Quantify Classification Uncertainty. Advances in Neural Information Processing Systems 31 (NeurIPS 2018). https://neurips.cc/virtual/2018/poster/11322

- [2] Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. Proceedings of the 33rd International Conference on Machine Learning (ICML 2016). arXiv:1506.02142.
- [3] Daxberger, E., Kristiadi, A., Immer, A., Eschenhagen, R., Bauer, M., & Hennig, P. (2021). Laplace Redux — Effortless Bayesian Deep Learning. Advances in Neural Information Processing Systems 34 (NeurIPS 2021). arXiv:2106.14806.
- [4] Maddox, W. J., Izmailov, P., Garipov, T., Vetrov, D. P., & Wilson, A. G. (2019). A Simple Baseline for Bayesian Uncertainty in Deep Learning. Advances in Neural Information Processing Systems 32 (NeurIPS 2019).
- [5] Bengs, V., Hüllermeier, E., & Waegeman, W. (2022). Pitfalls of Epistemic Uncertainty Quantification through Loss Minimisation. Advances in Neural Information Processing Systems 35 (NeurIPS 2022).
- [6] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. Proceedings of the IEEE, 86(11), 2278–2324.
- [7] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

13. Source-Code Webpage Link

The complete, reproducible source code (and this report) is published at:

https://github.com/Hou9123-cpu/edl-mnist